CSC 241/ COG 241
ABSTRACT DATA TYPES AND PROGRAMMING METHODOLOGY

Lecture01

Object-Oriented Programming(OOP) Intro

AGENDA

- Review for CSC 212
- OOP Intro
- File Management

REVIEW FOR CSC 212

”RE”START JAVA

- Java-development environment
  - JDK (Java SE Development Kit)
    - Download it (https://www.oracle.com/java/technologies/downloads/)
    - Check your OS!
  - JRE (Java Runtime Environment)
    - Not recommended for this class
- Java IDE
  - Lots of options:
    - Eclipse, NetBean, IntelliJ, …
  - Or plain texteditors?

HOW TO COMPILE JAVA IN TERMINAL

- To compile an application written in Java,
  - You can use java commands in terminal (for mac or Linux-based OS) or cmd (for windows)
- Commands
  - Compile:

    prompt: > `javac file_name.java`
  - Execute:

    prompt: > `java class_name`

PROGRAMMING EXERCISE

- Create `HelloWorld.java`
- Keys
  1) Read user's input by using `Scanner` class

OOP INTRO

STRUCTURED PROGRAMMING

- History
  - In 1950s, this programming paradigm was firstly shown in ALGOL 58.
  - In 1968, the name “structured programming” was used by Dr. Dijkstra.
  - In 1970s, it was developed in many languages and standardized.
- Procedure-based Programming
  - It aimed to run a program in a certain algorithmic or logical order by using defining procedures.
    - A program is driven by procedures.
    - Designed structures: selection(if/then), loop(while, for), recursion
  - Its main purpose was to examine algorithms or prove mathematical theories.
  - Dominant languages: ALGOL, BASIC, FORTRAN, Pascal, COBOL, C language

EXAMPLE OF STRUCTURED LANGUAGE

- C language
  - It was developed by Dennis Ritchie who was working at Bell Labs in the early 1970.
  - It was well-designed to control low-level machinery parts in a computer and flows of high-level processes.
    - It was able to organize the entire system of a computer.
    - Its main contribution was in controlling hardware.
  - It was used in building UNIX(and LINUX later)
  - Standardized by ANSI in 1989
  - Procedures are written in functions

NEEDS FOR NEW PROGRAMMING PARADIGM

- Issues in Structured Programming
  - It was difficult in relating data and procedures
  - It was too complicated as a program has more functions
- Solution: Object-Oriented Programming
  - New design for relating data and procedures
  - Introducing object
    - Data: properties belonging to an object
    - Procedures: behaviors or changes of data by an object
    - Functions are procedures performed by an object

HISTORY OF OOP

- In 1967, the concept was firstly used in SIMULA.
- In 1972, Smalltalk was released at Xerox PARC.
- There were more languages published, but not successful! WHY?
- The programming paradigm was organized in 1980s.
  - PCs were spread and the size of programs (plus data) were larger.
  - Perspective in developing machines has been expanded to software (or moved to software-centered.)
  - C++ and Object-C, which introduces "class", was published
  - Followed by more languages such as Python, Ruby, C#
- In 1995, Java was developed at Sun Microsystems (currently Oracle).

1967	SIMULA
1972	Smalltalk
1983	C++
1984	Object-C
1993	Python
1995	Ruby
1995	Java
2000	C#

HOW TO DEFINE AN OBJECT IN OOP

- Class
  - It is template of an object
  - It defines properties and/or behaviors of an object
  - Data of each property may represent state of an object
  - Behaviors may change states of an object
    - Called methods
- Example
  - A class *Person*

```
class Person {
   String name;
   int age, ssn;
        …
   Person(String name, int age, … ) {
      …
   }
   void gettingOld ( ) {
        age++;
   }
  void gettingOld (int years) {
        age += years;
   }
      …
}
```

PROPERTIES OF CLASS

- Class can *bundle* properties(data) and behaviors via **encapsulation**
  - Data can be hidden, eg., `ssn`.
  - Data may be accessed via methods if needed. e.g., `showSSN()`
- Class can *override* via **inheritance**
  - Class *`Student`* inherits class *`Person`*.
- Class can overload via Polymorphism.
  - *`gettingOld`*`()` and *`gettingOld`*`(`*`int years`*`)` have the same name but a different parameter

```
class Student extends Person {
  int id;
      …
  Student(String name, int age, …) {
    super(name, age, …);
    …
  }
    …
}
```

FILE MANAGEMENT

FILE CLASS

- In Java, a file can be instantiated by using File class
  - Java package

    `java.io.File`

  - Class document:

    https://docs.oracle.com/javase/7/docs/api/java/io/File.html

- Exception handling
  - File does not exist
- File path
  - `Path` class
  - `File.separator` (\ for windows, / for mac & linux)

PATH INTERFACE

- To use a full path of a file, interface Path can be used
  - Java package

    `java.nio.file`

  - Class document:

    https://docs.oracle.com/javase/8/docs/api/java/nio/file/Path.html

- How to build a file path
  - Final class `FileSystems`:

https://docs.oracle.com/javase/8/docs/api/java/nio/file/FileSystems.html

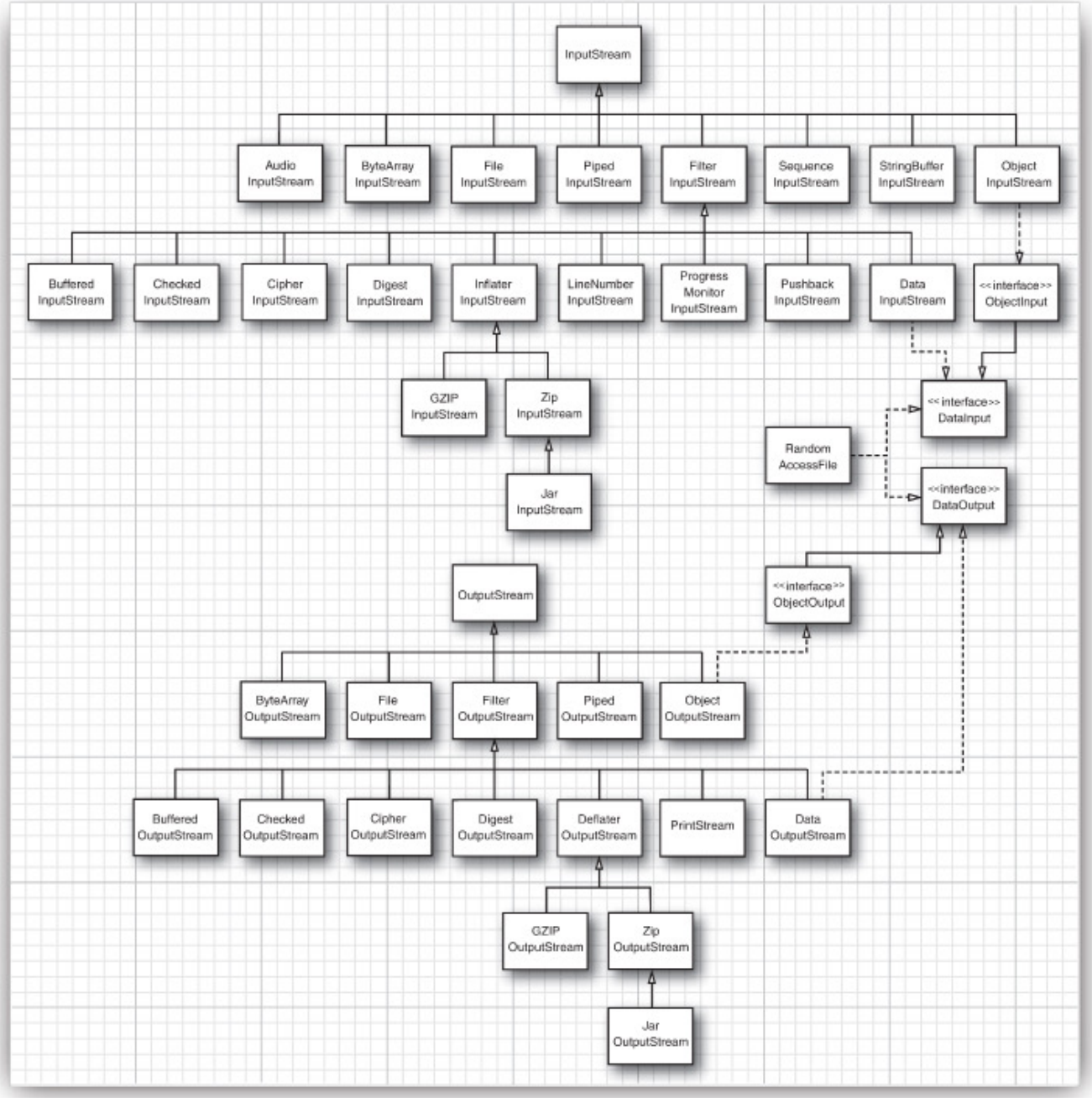
```
FileSystems.getDefault().getPath(file_name)
```

  - Final class Paths

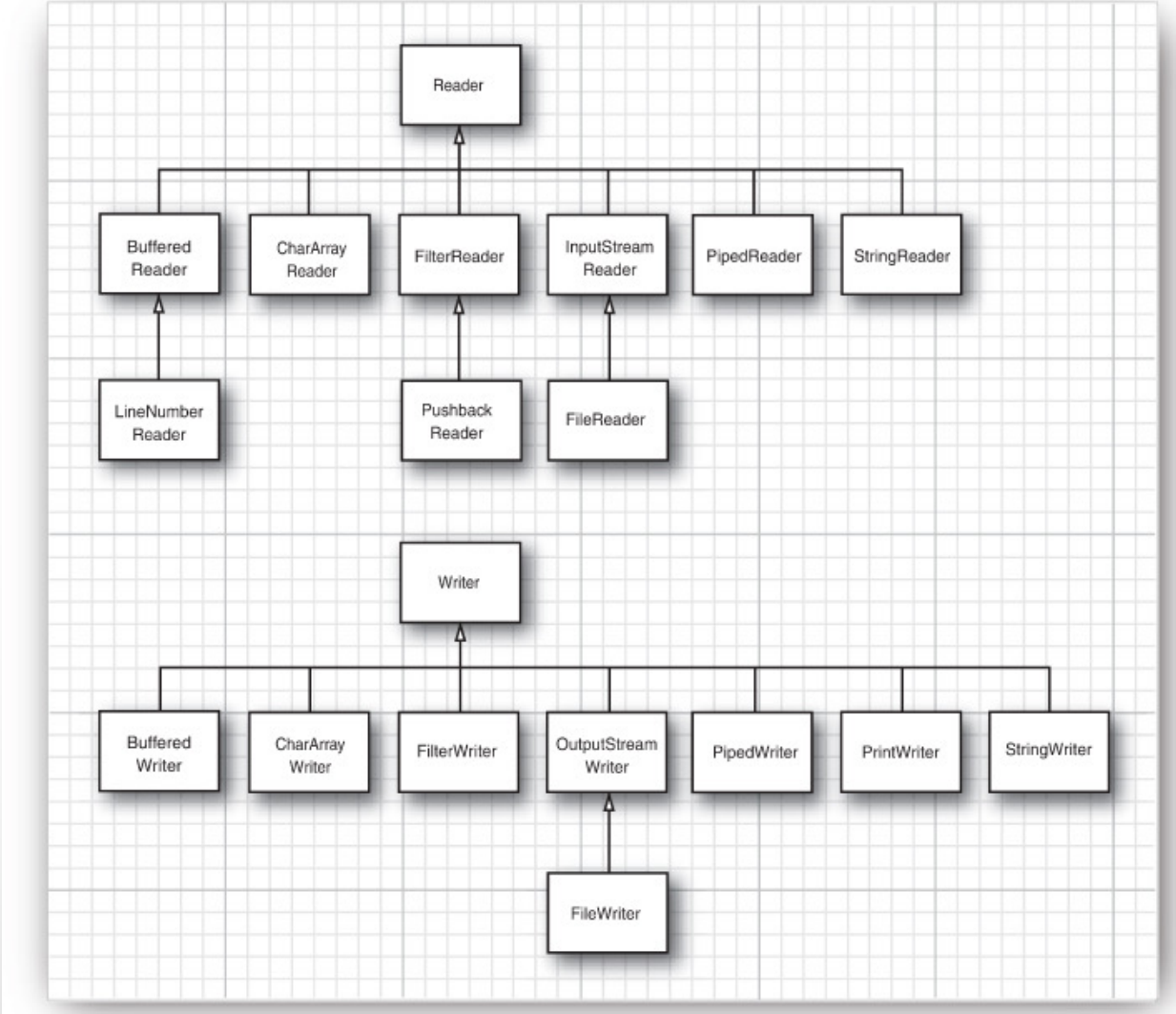
    https://docs.oracle.com/javase/7/docs/api/java/nio/file/Paths.html

```
Paths.get(System.getProperty("user.dir"), …, more)
```

INPUT AND OUTPUT STREAM HIERARCHY



READER AND WRITER HIERARCHY



HOW TO READ A FILE

- Use `Scanner`

```
File inputFile = new File(file_name);
Scanner line = new Scanner(inputFile);

While(line.hasNext()!= null)
{
    …
}
```

HOW TO READ A FILE(2)

- Use `BufferedReader`

```
File inputFile = new File(file_name);

FileInputStream fileInputStream =
newFileInputStream(inputFile);

InputStreamReader inputStreamReader = new
InputStreamReader(fileInputStream);

BufferedReader bufferedReader = new
BufferedReader(inputStreamReader);

While(bufferedReader.readLine() != null)
{
    …
}
```

HOW TO WRITE A FILE

- Use `BufferedWriter`

```
File outputFile = new File(file_name);

FileOutputStream fileOutputStream =
newFileOutputStream(outputFile);

OutputStreamWriter outputStreamReader = new
OutputStreamWriter(fileOutputtStream);

BufferedWriter bufferedWriter = new
BufferedWriter(outputStreamWriter);

While(bufferedReader.readLine() != null)
{
    bufferedWriter.write(new lines);
}
```

HOW TO PARSE A STRING

- A string is a character sequence.

String	S	U	N	Y		O	S	W	E	G	O
Index	0	1	2	3	4	5	6	7	8	9	10

- In this example,
  - The length of string: 11
  - The index of a delimiter (“ “): 4
- Two ways for paring a string
  - `String.substring(start,end);`
  - `String.split(delim);`

HOW TO PARSE A STRING

```
String str = “SUNY Oswego”;
```

- Use `String.substring`

```
String name, location;

name = str.substring(0,lastIndexOf(“ “));
location = str.substring(lastIndexOf(“ “)+1);
```

- Use `String.split`

```
String[] terms;

terms = str.split(“ “);
```

PROGRAMMING EXERCISE

- `MyClass.java`
- Keys
  1) Read a text file
  2) Replace a particular data in a text file
  3) Write a text file